

```
//+-----

//|          RectangleZoneAnalyzer_01_v2.mq4

//|

//|  Блок 01: поиск кандидатов зон и формирование zones.csv

//|

//|  Бычья зона: High-Low последнего медвежьего бара

//|  Медвежья зона: High-Low последнего бычьего бара

//|

//|  Фильтр расстояния применяется только между активными зонами

//|  одного типа. Анализ выполняется строго от старых баров к новым

//+-----
#property copyright "Copyright 2026"
#property version   "1.01"
#property strict
#property script_show_inputs

//--- Глубина истории. 0 = вся доступная история
input int HistoryBarsToScan = 100000;

//--- ATR, известный на момент подтверждения зоны
input ENUM_TIMEFRAMES ATRTimeframe = PERIOD_M5;
input int ATRPeriod = 14;

//--- Минимальный зазор между активными зонами одного типа
input double MinGapATR = 0.30;
input double MinGapPoints = 20.0;

//--- Пробитие зоны по закрытию свечи
input double BreakoutATR = 0.10;
input double BreakoutPoints = 10.0;

//--- Относительный тиковый объём
input int RelativeVolumePeriod = 20;

//--- Имя выходного файла в MQL4\Files
input string OutputFileName = "zones.csv";

//--- Печать прогресса каждые N обработанных баров. 0 = отключено
input int ProgressEveryBars = 5000;
```

```
//+-----  
//| Статистика одного бара
```

```
//+-----  
struct BarStats  
{  
    double open;  
    double high;  
    double low;  
    double close;  
  
    double rangePrice;  
    double bodyPrice;  
    double upperWickPrice;  
    double lowerWickPrice;  
  
    double rangeATR;  
    double bodyATR;  
    double bodyPercent;  
    double upperWickPercent;  
    double lowerWickPercent;  
  
    long    tickVolume;  
    double relativeVolume;  
};
```

```
//+-----  
//| Запись одного кандидата зоны
```

```
//+-----  
struct ZoneRecord  
{  
    string zoneID;  
    string symbol;  
    string timeframe;  
    string zoneType;  
  
    string candidateStatus;  
    string rejectReason;  
    string blockingZoneID;  
  
    datetime sourceTime;  
    datetime confirmBarTime;  
    datetime confirmationTime;  
    datetime rawBreakoutTime;  
    datetime brokenTime;  
    datetime endTime;
```

```

string finalStatus;

double zoneLow;
double zoneHigh;
double zoneHeightPrice;
double zoneHeightPoints;
double zoneHeightATR;

string atrTimeframe;
int atrPeriod;
double atrValue;

double minGapPointsInput;
double minGapATRInput;
double requiredGapPrice;
double requiredGapPoints;
double actualGapPrice;
double actualGapPoints;

double breakoutPointsInput;
double breakoutATRInput;
double breakoutThresholdPrice;
double breakoutThresholdPoints;

BarStats sourceBar;
BarStats confirmBar;

bool rawBreakout;
double rawBreakoutClose;

bool thresholdBreakout;
double breakoutClose;
double breakoutDistancePoints;
double breakoutDistanceATR;

double lifetimeMinutes;
double lifetimeHours;
int lifetimeBars;

//--- Внутренние поля, в CSV не выводятся
int confirmationDecisionShift;
};

ZoneRecord g_zones[];
int g_zoneCount = 0;

int g_activeZoneIndexes[];
int g_activeCount = 0;

int g_totalCandidates = 0;
int g_acceptedZones = 0;
int g_rejectedZones = 0;

```

```

int g_acceptedBull      = 0;
int g_acceptedBear      = 0;
int g_brokenZones       = 0;
int g_endActiveZones    = 0;

int g_totalBars = 0;
int g_atrTf      = 0;

//+-----+

//| Вспомогательные функции

//+-----+
string TimeframeToString(int timeframe)
{
    int tf = timeframe;
    if(tf == PERIOD_CURRENT || tf == 0)
        tf = Period();

    switch(tf)
    {
        case PERIOD_M1:    return("M1");
        case PERIOD_M5:    return("M5");
        case PERIOD_M15:   return("M15");
        case PERIOD_M30:   return("M30");
        case PERIOD_H1:    return("H1");
        case PERIOD_H4:    return("H4");
        case PERIOD_D1:    return("D1");
        case PERIOD_W1:    return("W1");
        case PERIOD_MN1:   return("MN1");
    }

    return("TF" + IntegerToString(tf));
}

string TimeToCsv(datetime value)
{
    if(value <= 0)
        return("");

    return(TimeToString(value, TIME_DATE | TIME_MINUTES));
}

string TimeToID(datetime value)
{
    string result = TimeToString(value, TIME_DATE | TIME_MINUTES);

    StringReplace(result, ".", "");
    StringReplace(result, ":", "");
    StringReplace(result, " ", "_");
}

```

```

    return(result);
}

string CleanIdentifier(string value)
{
    StringReplace(value, " ", "_");
    StringReplace(value, ";", "_");
    StringReplace(value, "/", "_");
    StringReplace(value, "\\ ", "_");
    StringReplace(value, ":", "_");

    return(value);
}

string BuildZoneID(
    string zoneType,
    datetime sourceTime,
    datetime confirmationTime
)
{
    return(
        CleanIdentifier(Symbol()) + "_" +
        TimeframeToString(Period()) + "_" +
        zoneType + "_" +
        TimeToID(sourceTime) + "_" +
        TimeToID(confirmationTime)
    );
}

string CsvField(string value)
{
    bool needQuotes = false;

    if(StringFind(value, ";") >= 0 ||
        StringFind(value, "\"") >= 0 ||
        StringFind(value, "\r") >= 0 ||
        StringFind(value, "\n") >= 0)
    {
        needQuotes = true;
    }

    if(!needQuotes)
        return(value);

    StringReplace(value, "\"", "\\\"");
    return("\"" + value + "\"");
}

string PriceToString(double value)
{
    return(DoubleToString(value, Digits));
}

```

```

string DoubleOrBlank(double value, int decimals = 6)
{
    if(value < 0.0)
        return("");

    return(DoubleToString(value, decimals));
}

string PriceOrBlank(double value)
{
    if(value < 0.0)
        return("");

    return(DoubleToString(value, Digits));
}

string IntOrBlank(int value)
{
    if(value < 0)
        return("");

    return(IntegerToString(value));
}

string LongToCsv(long value)
{
    return(DoubleToString((double)value, 0));
}

void AppendField(string &line, string value, bool first = false)
{
    if(!first)
        line += ";";

    line += CsvField(value);
}

//+-----
//| Относительный объём бара
//+-----
double GetRelativeVolume(int shift, int period)
{
    if(period <= 0)
        return(-1.0);

    if(shift < 0 || shift >= g_totalBars)
        return(-1.0);
}

```

```

if(shift + period >= g_totalBars)
    return(-1.0);

double sum = 0.0;

for(int i = 1; i <= period; i++)
{
    long volume = iVolume(NULL, 0, shift + i);
    sum += (double)volume;
}

double average = sum / period;

if(average <= 0.0)
    return(-1.0);

return((double)iVolume(NULL, 0, shift) / average);
}

//+-----

//| Заполнение статистики бара

//+-----
void FillBarStats(int shift, double atrValue, BarStats &stats)
{
    stats.open    = iOpen(NULL, 0, shift);
    stats.high    = iHigh(NULL, 0, shift);
    stats.low     = iLow(NULL, 0, shift);
    stats.close   = iClose(NULL, 0, shift);

    stats.rangePrice = stats.high - stats.low;
    stats.bodyPrice  = MathAbs(stats.close - stats.open);

    stats.upperWickPrice =
        stats.high - MathMax(stats.open, stats.close);

    stats.lowerWickPrice =
        MathMin(stats.open, stats.close) - stats.low;

    if(stats.upperWickPrice < 0.0)
        stats.upperWickPrice = 0.0;

    if(stats.lowerWickPrice < 0.0)
        stats.lowerWickPrice = 0.0;

    if(atrValue > 0.0)
    {
        stats.rangeATR = stats.rangePrice / atrValue;
        stats.bodyATR  = stats.bodyPrice / atrValue;
    }
}

```

```

else
{
    stats.rangeATR = -1.0;
    stats.bodyATR  = -1.0;
}

if(stats.rangePrice > 0.0)
{
    stats.bodyPercent =
        100.0 * stats.bodyPrice / stats.rangePrice;

    stats.upperWickPercent =
        100.0 * stats.upperWickPrice / stats.rangePrice;

    stats.lowerWickPercent =
        100.0 * stats.lowerWickPrice / stats.rangePrice;
}
else
{
    stats.bodyPercent      = 0.0;
    stats.upperWickPercent = 0.0;
    stats.lowerWickPercent = 0.0;
}

stats.tickVolume = iVolume(NULL, 0, shift);
stats.relativeVolume =
    GetRelativeVolume(shift, RelativeVolumePeriod);
}

//+-----

//| Плановое время закрытия бара

//+-----
datetime GetBarCloseTime(int timeframe, int shift)
{
    datetime barOpenTime = iTime(NULL, timeframe, shift);

    if(barOpenTime <= 0)
        return(0);

    int seconds = PeriodSeconds((ENUM_TIMEFRAMES)timeframe);

    if(seconds <= 0)
        return(0);

    return(barOpenTime + seconds);
}

//+-----

```

```
///  
//| ATR, полностью известный к decisionTime
```

```
///  
//+-----
```

```
double GetClosedATRAt(datetime decisionTime)
```

```
{  
    if(ATRPeriod <= 0 || decisionTime <= 0)  
        return(0.0);
```

```
    int atrBars = iBars(NULL, g_atrTf);
```

```
    if(atrBars <= ATRPeriod + 2)  
        return(0.0);
```

```
    int atrShift = iBarShift(  
        NULL,  
        g_atrTf,  
        decisionTime,  
        false  
    );
```

```
    if(atrShift < 0)  
        return(0.0);
```

```
    /*
```

Если decisionTime попал внутрь ещё не закрытого ATR-бара,
используем предыдущий бар.

Если decisionTime находится в торговом разрыве, iBarShift()

возвращает последний существующий бар перед разрывом. Если его

плановое время закрытия уже наступило, этот бар использовать не

```
*/
```

```
datetime atrBarCloseTime =  
    GetBarCloseTime(g_atrTf, atrShift);
```

```
if(atrBarCloseTime <= 0)  
    return(0.0);
```

```
if(atrBarCloseTime > decisionTime)  
    atrShift++;
```

```
if(atrShift >= atrBars)  
    return(0.0);
```

```
double atrValue = iATR(  
    NULL,  
    g_atrTf,  
    ATRPeriod,  
    atrShift
```

```

);

if(atrValue == EMPTY_VALUE || atrValue <= 0.0)
    return(0.0);

return(atrValue);
}

```

```

//+-----
//| Расстояние между ближайшими границами зон

```

```

//+-----
double GetZonesGap(
    double firstLow,
    double firstHigh,
    double secondLow,
    double secondHigh
)
{
    if(firstLow > secondHigh)
        return(firstLow - secondHigh);

    if(secondLow > firstHigh)
        return(secondLow - firstHigh);

    return(0.0);
}

```

```

//+-----
//| Добавление активной зоны

```

```

//+-----
void AddActiveZone(int zoneIndex)
{
    ArrayResize(g_activeZoneIndexes, g_activeCount + 1);
    g_activeZoneIndexes[g_activeCount] = zoneIndex;
    g_activeCount++;
}

```

```

//+-----
//| Удаление элемента активного массива

```

```

//+-----
void RemoveActivePosition(int position)
{
    if(position < 0 || position >= g_activeCount)

```

```

        return;

    for(int i = position; i < g_activeCount - 1; i++)
        g_activeZoneIndexes[i] = g_activeZoneIndexes[i + 1];

    g_activeCount--;
    ArrayResize(g_activeZoneIndexes, g_activeCount);
}

//+-----
//| Завершение жизненного цикла зоны
//+-----
void FinalizeZone(
    int zoneIndex,
    string finalStatus,
    datetime endTime,
    int endDecisionShift
)
{
    g_zones[zoneIndex].finalStatus = finalStatus;
    g_zones[zoneIndex].endTime      = endTime;

    double seconds =
        (double) (endTime - g_zones[zoneIndex].confirmationTime);

    if(seconds < 0.0)
        seconds = 0.0;

    g_zones[zoneIndex].lifetimeMinutes = seconds / 60.0;
    g_zones[zoneIndex].lifetimeHours   = seconds / 3600.0;

    int bars =
        g_zones[zoneIndex].confirmationDecisionShift -
        endDecisionShift;

    if(bars < 0)
        bars = 0;

    g_zones[zoneIndex].lifetimeBars = bars;
}

//+-----
//| Проверка активных зон закрытием текущего исторического бара
//+-----
void ProcessBreakoutsOnBar(
    int closedBarShift,

```

```

datetime decisionTime
)
{
double closePrice = iClose(NULL, 0, closedBarShift);

int position = 0;

while(position < g_activeCount)
{
    int zoneIndex = g_activeZoneIndexes[position];

    bool bullish =
        (g_zones[zoneIndex].zoneType == "BULL");

    bool rawBroken = false;
    bool thresholdBroken = false;

    if(bullish)
    {
        rawBroken =
            (closePrice < g_zones[zoneIndex].zoneLow);

        thresholdBroken =
            (closePrice <
             g_zones[zoneIndex].zoneLow -
             g_zones[zoneIndex].breakoutThresholdPrice);
    }
    else
    {
        rawBroken =
            (closePrice > g_zones[zoneIndex].zoneHigh);

        thresholdBroken =
            (closePrice >
             g_zones[zoneIndex].zoneHigh +
             g_zones[zoneIndex].breakoutThresholdPrice);
    }

    if(rawBroken && !g_zones[zoneIndex].rawBreakout)
    {
        g_zones[zoneIndex].rawBreakout      = true;
        g_zones[zoneIndex].rawBreakoutTime   = decisionTime;
        g_zones[zoneIndex].rawBreakoutClose  = closePrice;
    }

    if(thresholdBroken)
    {
        g_zones[zoneIndex].thresholdBreakout = true;
        g_zones[zoneIndex].brokenTime        = decisionTime;
        g_zones[zoneIndex].breakoutClose     = closePrice;

        double breakoutDistancePrice;
    }
}

```

```

        if(bullish)
        {
            breakoutDistancePrice =
                g_zones[zoneIndex].zoneLow - closePrice;
        }
        else
        {
            breakoutDistancePrice =
                closePrice - g_zones[zoneIndex].zoneHigh;
        }

        if(breakoutDistancePrice < 0.0)
            breakoutDistancePrice = 0.0;

        g_zones[zoneIndex].breakoutDistancePoints =
            breakoutDistancePrice / Point;

        if(g_zones[zoneIndex].atrValue > 0.0)
        {
            g_zones[zoneIndex].breakoutDistanceATR =
                breakoutDistancePrice /
                    g_zones[zoneIndex].atrValue;
        }
        else
        {
            g_zones[zoneIndex].breakoutDistanceATR = -1.0;
        }

        FinalizeZone(
            zoneIndex,
            "BROKEN",
            decisionTime,
            closedBarShift - 1
        );

        g_brokenZones++;
        RemoveActivePosition(position);
        continue;
    }

    position++;
}

//+-----

//| Создание записи кандидата

//+-----
int AddCandidate(
    int sourceShift,

```

```

    int confirmShift,
    string zoneType
)
{
    int index = g_zoneCount;

    ArrayResize(g_zones, g_zoneCount + 1);
    g_zoneCount++;

    datetime sourceTime =
        iTime(NULL, 0, sourceShift);

    datetime confirmBarTime =
        iTime(NULL, 0, confirmShift);

    datetime confirmationTime =
        GetBarCloseTime(Period(), confirmShift);

    double atrValue =
        GetClosedATRAt(confirmationTime);

    FillBarStats(
        sourceShift,
        atrValue,
        g_zones[index].sourceBar
    );

    FillBarStats(
        confirmShift,
        atrValue,
        g_zones[index].confirmBar
    );

    g_zones[index].zoneID =
        BuildZoneID(zoneType, sourceTime, confirmationTime);

    g_zones[index].symbol      = Symbol();
    g_zones[index].timeframe   = TimeframeToString(Period());
    g_zones[index].zoneType    = zoneType;

    g_zones[index].candidateStatus = "";
    g_zones[index].rejectReason    = "";
    g_zones[index].blockingZoneID  = "";

    g_zones[index].sourceTime      = sourceTime;
    g_zones[index].confirmBarTime  = confirmBarTime;
    g_zones[index].confirmationTime = confirmationTime;
    g_zones[index].rawBreakoutTime = 0;
    g_zones[index].brokenTime      = 0;
    g_zones[index].endTime         = 0;

    g_zones[index].finalStatus = "";

```

```

g_zones[index].zoneLow    =
    g_zones[index].sourceBar.low;

g_zones[index].zoneHigh   =
    g_zones[index].sourceBar.high;

g_zones[index].zoneHeightPrice =
    g_zones[index].zoneHigh -
    g_zones[index].zoneLow;

g_zones[index].zoneHeightPoints =
    g_zones[index].zoneHeightPrice / Point;

if(atrValue > 0.0)
{
    g_zones[index].zoneHeightATR =
        g_zones[index].zoneHeightPrice / atrValue;
}
else
{
    g_zones[index].zoneHeightATR = -1.0;
}

g_zones[index].atrTimeframe =
    TimeframeToString(g_atrTf);

g_zones[index].atrPeriod = ATRPeriod;
g_zones[index].atrValue  = atrValue;

g_zones[index].minGapPointsInput = MinGapPoints;
g_zones[index].minGapATRInput    = MinGapATR;

double fixedGap = MinGapPoints * Point;
double atrGap   = atrValue * MinGapATR;

g_zones[index].requiredGapPrice =
    MathMax(fixedGap, atrGap);

g_zones[index].requiredGapPoints =
    g_zones[index].requiredGapPrice / Point;

g_zones[index].actualGapPrice  = -1.0;
g_zones[index].actualGapPoints = -1.0;

g_zones[index].breakoutPointsInput = BreakoutPoints;
g_zones[index].breakoutATRInput    = BreakoutATR;

double fixedBreakout = BreakoutPoints * Point;
double atrBreakout   = atrValue * BreakoutATR;

g_zones[index].breakoutThresholdPrice =
    MathMax(fixedBreakout, atrBreakout);

```

```

g_zones[index].breakoutThresholdPoints =
    g_zones[index].breakoutThresholdPrice / Point;

g_zones[index].rawBreakout      = false;
g_zones[index].rawBreakoutClose = 0.0;

g_zones[index].thresholdBreakout = false;
g_zones[index].breakoutClose     = 0.0;
g_zones[index].breakoutDistancePoints = -1.0;
g_zones[index].breakoutDistanceATR  = -1.0;

g_zones[index].lifetimeMinutes = -1.0;
g_zones[index].lifetimeHours   = -1.0;
g_zones[index].lifetimeBars    = -1;

g_zones[index].confirmationDecisionShift =
    confirmShift - 1;

return(index);
}

//+-----+
//| Проверка расстояния и принятие/отклонение кандидата
//+-----+
void EvaluateCandidateDistance(int candidateIndex)
{
    double minimumGap = 1.0e100;
    int nearestZoneIndex = -1;

    for(int i = 0; i < g_activeCount; i++)
    {
        int activeIndex = g_activeZoneIndexes[i];

        if(g_zones[activeIndex].zoneType !=
            g_zones[candidateIndex].zoneType)
        {
            continue;
        }

        double gap = GetZonesGap(
            g_zones[candidateIndex].zoneLow,
            g_zones[candidateIndex].zoneHigh,
            g_zones[activeIndex].zoneLow,
            g_zones[activeIndex].zoneHigh
        );

        if(gap < minimumGap)
        {
            minimumGap = gap;

```

```

        nearestZoneIndex = activeIndex;
    }
}

if(nearestZoneIndex >= 0)
{
    g_zones[candidateIndex].actualGapPrice = minimumGap;
    g_zones[candidateIndex].actualGapPoints =
        minimumGap / Point;
}

if(nearestZoneIndex >= 0 &&
    minimumGap < g_zones[candidateIndex].requiredGapPrice)
{
    g_zones[candidateIndex].candidateStatus =
        "REJECTED_DISTANCE";

    g_zones[candidateIndex].rejectReason =
        "MIN_GAP";

    g_zones[candidateIndex].blockingZoneID =
        g_zones[nearestZoneIndex].zoneID;

    g_zones[candidateIndex].finalStatus =
        "REJECTED";

    g_rejectedZones++;
    return;
}

g_zones[candidateIndex].candidateStatus = "ACCEPTED";
g_zones[candidateIndex].finalStatus      = "ACTIVE";

AddActiveZone(candidateIndex);

g_acceptedZones++;

if(g_zones[candidateIndex].zoneType == "BULL")
    g_acceptedBull++;
else
    g_acceptedBear++;
}

//+-----

//| Проверка разворотного паттерна

//+-----
void DetectCandidateOnBar(int confirmShift)
{
    int sourceShift = confirmShift + 1;

```

```

double sourceOpen  = iOpen(NULL, 0, sourceShift);
double sourceClose = iClose(NULL, 0, sourceShift);

double confirmOpen  = iOpen(NULL, 0, confirmShift);
double confirmClose = iClose(NULL, 0, confirmShift);

bool bullishCandidate =
    sourceClose < sourceOpen &&
    confirmClose > confirmOpen &&
    confirmClose > sourceOpen;

bool bearishCandidate =
    sourceClose > sourceOpen &&
    confirmClose < confirmOpen &&
    confirmClose < sourceOpen;

if(bullishCandidate)
{
    int candidateIndex =
        AddCandidate(sourceShift, confirmShift, "BULL");

    g_totalCandidates++;
    EvaluateCandidateDistance(candidateIndex);
}

if(bearishCandidate)
{
    int candidateIndex =
        AddCandidate(sourceShift, confirmShift, "BEAR");

    g_totalCandidates++;
    EvaluateCandidateDistance(candidateIndex);
}
}

//+-----

//| Завершение оставшихся активных зон

//+-----
void FinalizeActiveZonesAtHistoryEnd()
{
    // История обработана по последний закрытый бар shift=1.

    // Используем его плановое время закрытия, а не время открытия

    // следующего реально существующего бара после возможного разрыва.
    datetime endTime = GetBarCloseTime(Period(), 1);

    for(int i = 0; i < g_activeCount; i++)
    {

```

```

        int zoneIndex = g_activeZoneIndexes[i];

        FinalizeZone(
            zoneIndex,
            "END_OF_HISTORY",
            endTime,
            0
        );

        g_endActiveZones++;
    }
}

//+-----

//| Запись заголовка CSV

//+-----
void WriteCsvHeader(int handle)
{
    string header =

    "ZoneID;Symbol;Timeframe;ZoneType;CandidateStatus;RejectReason;Block
    "SourceTime;ConfirmBarTime;ConfirmationTime;RawBreakoutTime;BrokenT
    "ZoneLow;ZoneHigh;ZoneHeightPrice;ZoneHeightPoints;ZoneHeightATR;"
    "ATRTIMEFRAME;ATRPeriod;ATRValue;MinGapPoints;MinGapATR;RequiredGap
    "BreakoutPoints;BreakoutATR;BreakoutThresholdPrice;BreakoutThreshold
    "SourceOpen;SourceHigh;SourceLow;SourceClose;SourceRangeATR;SourceB
    "ConfirmOpen;ConfirmHigh;ConfirmLow;ConfirmClose;ConfirmRangeATR;Con
    "RawBreakout;RawBreakoutClose;ThresholdBreakout;BreakoutClose;Breako
        "LifetimeMinutes;LifetimeHours;LifetimeBars";

    FileWriteString(handle, header + "\r\n");
}

//+-----

//| Запись одной строки CSV

//+-----
void WriteZoneRow(int handle, ZoneRecord &zone)
{
    string line = "";

```

```

AppendField(line, zone.zoneID, true);
AppendField(line, zone.symbol);
AppendField(line, zone.timeframe);
AppendField(line, zone.zoneType);
AppendField(line, zone.candidateStatus);
AppendField(line, zone.rejectReason);
AppendField(line, zone.blockingZoneID);

AppendField(line, TimeToCsv(zone.sourceTime));
AppendField(line, TimeToCsv(zone.confirmBarTime));
AppendField(line, TimeToCsv(zone.confirmationTime));
AppendField(line, TimeToCsv(zone.rawBreakoutTime));
AppendField(line, TimeToCsv(zone.brokenTime));
AppendField(line, TimeToCsv(zone.endTime));
AppendField(line, zone.finalStatus);

AppendField(line, PriceToString(zone.zoneLow));
AppendField(line, PriceToString(zone.zoneHigh));
AppendField(line, PriceToString(zone.zoneHeightPrice));
AppendField(line, DoubleToString(zone.zoneHeightPoints, 2));
AppendField(line, DoubleOrBlank(zone.zoneHeightATR, 6));

AppendField(line, zone.atrTimeframe);
AppendField(line, IntegerToString(zone.atrPeriod));
AppendField(line, PriceToString(zone.atrValue));
AppendField(line, DoubleToString(zone.minGapPointsInput, 2
));
AppendField(line, DoubleToString(zone.minGapATRInput, 6));
AppendField(line, PriceToString(zone.requiredGapPrice));
AppendField(line, DoubleToString(zone.requiredGapPoints, 2
));
AppendField(line, PriceOrBlank(zone.actualGapPrice));
AppendField(line, DoubleOrBlank(zone.actualGapPoints, 2));

AppendField(line, DoubleToString(zone.breakoutPointsInput, 2
));
AppendField(line, DoubleToString(zone.breakoutATRInput, 6));
AppendField(line, PriceToString(zone.breakoutThresholdPrice
));
AppendField(line, DoubleToString(
zone.breakoutThresholdPoints, 2));

AppendField(line, PriceToString(zone.sourceBar.open));
AppendField(line, PriceToString(zone.sourceBar.high));
AppendField(line, PriceToString(zone.sourceBar.low));
AppendField(line, PriceToString(zone.sourceBar.close));
AppendField(line, DoubleOrBlank(zone.sourceBar.rangeATR, 6
));
AppendField(line, DoubleOrBlank(zone.sourceBar.bodyATR, 6));
AppendField(line, DoubleToString(zone.sourceBar.bodyPercent,
4));

```

```

AppendField(line, DoubleToString(
zone.sourceBar.upperWickPercent, 4));
AppendField(line, DoubleToString(
zone.sourceBar.lowerWickPercent, 4));
AppendField(line, LongToCsv(zone.sourceBar.tickVolume));
AppendField(line, DoubleOrBlank(
zone.sourceBar.relativeVolume, 6));

AppendField(line, PriceToString(zone.confirmBar.open));
AppendField(line, PriceToString(zone.confirmBar.high));
AppendField(line, PriceToString(zone.confirmBar.low));
AppendField(line, PriceToString(zone.confirmBar.close));
AppendField(line, DoubleOrBlank(zone.confirmBar.rangeATR, 6
));
AppendField(line, DoubleOrBlank(zone.confirmBar.bodyATR, 6
));
AppendField(line, DoubleToString(zone.confirmBar.bodyPercent
, 4));
AppendField(line, DoubleToString(
zone.confirmBar.upperWickPercent, 4));
AppendField(line, DoubleToString(
zone.confirmBar.lowerWickPercent, 4));
AppendField(line, LongToCsv(zone.confirmBar.tickVolume));
AppendField(line, DoubleOrBlank(
zone.confirmBar.relativeVolume, 6));

AppendField(line, zone.rawBreakout ? "1" : "0");
AppendField(
    line,
    zone.rawBreakout ? PriceToString(zone.rawBreakoutClose) :
""
);

AppendField(line, zone.thresholdBreakout ? "1" : "0");
AppendField(
    line,
    zone.thresholdBreakout ? PriceToString(zone.breakoutClose
) : ""
);

AppendField(
    line,
    DoubleOrBlank(zone.breakoutDistancePoints, 2)
);

AppendField(
    line,
    DoubleOrBlank(zone.breakoutDistanceATR, 6)
);

AppendField(line, DoubleOrBlank(zone.lifetimeMinutes, 2));
AppendField(line, DoubleOrBlank(zone.lifetimeHours, 6));

```

```

AppendField(line, IntOrBlank(zone.lifetimeBars));

FileWriteString(handle, line + "\r\n");
}

//+-----
//| Сохранение zones.csv
//+-----
bool SaveZonesCsv()
{
    string fileName = OutputFileName;

    if(StringLen(fileName) == 0)
    {
        fileName =
            "zones_" +
            CleanIdentifier(Symbol()) + "_" +
            TimeframeToString(Period()) + ".csv";
    }

    ResetLastError();

    int handle = FileOpen(
        fileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка открытия файла ",
            fileName,
            ". Код ошибки: ",
            GetLastError()
        );

        ResetLastError();
        return(false);
    }

    WriteCsvHeader(handle);

    for(int i = 0; i < g_zoneCount; i++)
        WriteZoneRow(handle, g_zones[i]);

    FileFlush(handle);
    FileClose(handle);

    string fullPath =

```

```

        TerminalInfoString(TERMINAL_DATA_PATH) +
        "\\MQL4\\Files\\" +
        fileName;

    Print("Файл создан: ", fullPath);

    return(true);
}

//+-----+
//| Проверка входных параметров
//+-----+
bool ValidateInputs()
{
    if(HistoryBarsToScan < 0)
    {
        Print(
"Ошибка: HistoryBarsToScan не может быть отрицательным.");
        return(false);
    }

    if(ATRPeriod < 1)
    {
        Print("Ошибка: ATRPeriod должен быть больше 0.");
        return(false);
    }

    if(MinGapATR < 0.0 || MinGapPoints < 0.0)
    {
        Print(
"Ошибка: параметры минимального расстояния не могут быть отрицательными.");
        return(false);
    }

    if(BreakoutATR < 0.0 || BreakoutPoints < 0.0)
    {
        Print(
"Ошибка: параметры пробития не могут быть отрицательными.");
        return(false);
    }

    if(RelativeVolumePeriod < 1)
    {
        Print(
"Ошибка: RelativeVolumePeriod должен быть больше 0.");
        return(false);
    }

    if(ProgressEveryBars < 0)

```

```

    {
        Print(
"Ошибка: ProgressEveryBars не может быть отрицательным.");
        return(false);
    }

    return(true);
}

//+-----

//| Главная функция скрипта

//+-----
void OnStart()
{
    if(!ValidateInputs())
        return;

    g_totalBars = iBars(NULL, 0);

    if(g_totalBars < 4)
    {
        Print("Недостаточно истории: найдено баров ", g_totalBars
);
        return;
    }

    g_atrTf = (int)ATRTimeframe;

    if(g_atrTf == PERIOD_CURRENT || g_atrTf == 0)
        g_atrTf = Period();

    ArrayResize(g_zones, 0);
    ArrayResize(g_activeZoneIndexes, 0);

    g_zoneCount          = 0;
    g_activeCount         = 0;
    g_totalCandidates     = 0;
    g_acceptedZones       = 0;
    g_rejectedZones       = 0;
    g_acceptedBull         = 0;
    g_acceptedBear         = 0;
    g_brokenZones         = 0;
    g_endActiveZones      = 0;

    int availableConfirmBars = g_totalBars - 2;
    int firstConfirmShift;

    if(HistoryBarsToScan == 0)
        firstConfirmShift = availableConfirmBars;

```

```

else
    firstConfirmShift =
        (HistoryBarsToScan < availableConfirmBars)
        ? HistoryBarsToScan
        : availableConfirmBars;

if(firstConfirmShift < 1)
{
    Print("Нет закрытых баров для анализа.");
    return;
}

Print(
"=====
");
Print("RectangleZoneAnalyzer_01");
Print("Символ: ", Symbol(), " ТФ: ", TimeframeToString(
Period()));
Print("Баров для анализа: ", firstConfirmShift);
Print("ATR: ", TimeframeToString(g_atrTf), "(", ATRPeriod,
")");
Print(
"=====
");

int processed = 0;

//--- Строго хронологический проход: от старых баров к новым
for(int confirmShift = firstConfirmShift;
    confirmShift >= 1;
    confirmShift--)
{
    if(IsStopped())
    {
        Print("Выполнение остановлено пользователем.");
        return;
    }

    datetime decisionTime =
        GetBarCloseTime(Period(), confirmShift);

    // 1. Сначала закрытием confirmShift проверяем пробитие
    // уже существовавших зон.
    ProcessBreakoutsOnBar(
        confirmShift,
        decisionTime
    );

    // 2. После закрытия этого бара может подтвердиться новая зона.
    DetectCandidateOnBar(confirmShift);

    processed++;
}

```

```

if(ProgressEveryBars > 0 &&
    (processed % ProgressEveryBars == 0 ||
     processed == firstConfirmShift))
{
    double percent =
        100.0 * processed / firstConfirmShift;

    Print(
        "Прогресс: ",
        processed,
        "/",
        firstConfirmShift,
        " (",
        DoubleToString(percent, 1),
        "%)",
        " кандидатов=",
        g_totalCandidates,
        " активных=",
        g_activeCount
    );
}

FinalizeActiveZonesAtHistoryEnd();

bool saved = SaveZonesCsv();

Print(
"=====
");
);
Print("РЕЗУЛЬТАТ АНАЛИЗА ЗОН");
Print("Всего кандидатов      : ", g_totalCandidates);
Print("Принято зон             : ", g_acceptedZones);
Print("Отклонено по расстоянию: ", g_rejectedZones);
Print("Бычьих принято           : ", g_acceptedBull);
Print("Медвежьих принято        : ", g_acceptedBear);
Print("Пробито                   : ", g_brokenZones);
Print("Активны в конце истории: ", g_endActiveZones);
Print("Строк в zones.csv         : ", g_zoneCount);
Print("Сохранение файла         : ", saved ? "OK" : "ERROR");
Print(
"=====
");
);
}

//+-----

```